

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ЛАБОРАТОРНАЯ РАБОТА № 3

По дисциплине: «Frontend разработка»

Тема: «Разработка одностраничного веб-приложения (SPA) с использованием фреймворка
Vue.js»

Вариант: «Платформа для онлайн-курсов и обучения»

Выполнил:
студент группы **J3210**
Райков Илья Алексеевич

Содержание

1	Цель и задачи работы	2
2	Архитектура проекта	2
2.1	Файловая структура (src/)	2
3	Описание реализации	2
3.1	Компонентный подход	2
3.2	Маршрутизация и безопасность	3
3.3	Использование Composables	3
3.4	Взаимодействие с API	3
4	Листинги основного кода	3
4.1	Реализация Composable (useTheme.js)	3
4.2	Глобальный стор авторизации (stores/auth.js)	3
4.3	Интеграция Axios Interceptors (api/index.js)	4
5	Заключение	4

1 Цель и задачи работы

Цель лабораторной работы: освоить современный стек Frontend-разработки на базе фреймворка Vue 3. Мигрировать ранее разработанное приложение на архитектуру Single Page Application (SPA), реализовать компонентный подход, управление состоянием через Pinia и маршрутизацию через Vue Router.

Задачи:

- Перенести логику приложения на фреймворк Vue 3 с использованием Composition API.
- Разработать систему маршрутизации (маршруты для каталога, авторизации и личного кабинета).
- Реализовать работу с внешним API посредством библиотеки `axios`, включая настройку перехватчиков (interceptors) для передачи токенов.
- Выполнить декомпозицию интерфейса на переиспользуемые компоненты.
- Использовать механизм `composables` для выделения логики темизации сайта в отдельный функциональный блок.
- Настроить глобальное хранилище данных (Pinia) для управления сессией пользователя.

2 Архитектура проекта

Проект переведен на компонентно-ориентированную структуру, типичную для современных Vue-приложений.

2.1 Файловая структура (src/)

```
src/
├── api/           # Настройка axios и сервисный слой API
├── assets/        # CSS стили (мигрированы из LP1/LP2)
├── components/    # Переиспользуемые компоненты (NavBar, CourseCard,
├── composables/   # Вынесенная логика (useTheme.js)
├── layouts/       # Общие обёртки страниц (MainLayout.vue)
├── router/        # Конфигурация Vue Router и защита маршрутов
├── stores/        # Глобальное состояние Pinia (auth.js)
├── views/         # Страницы приложения (HomePage, LoginPage и др.)
└── App.vue       # Корневой компонент
```

3 Описание реализации

3.1 Компонентный подход

Приложение разделено на функциональные блоки. Компонент `CourseCard.vue` является универсальным: он принимает данные курса через `props` и адаптирует свой вид (отображение кнопки покупки или прогресс-бара) в зависимости от контекста (каталог или личный кабинет). Компонент `NavBar.vue` реактивно реагирует на изменения в `authStore`, отображая соответствующие элементы управления для гостей и авторизованных пользователей.

3.2 Маршрутизация и безопасность

С помощью Vue Router настроена навигация без перезагрузки страницы. Реализован механизм Navigation Guards: доступ к странице «Личный кабинет» (/dashboard) автоматически блокируется для неавторизованных пользователей с перенаправлением на страницу входа.

3.3 Использование Composables

Согласно требованиям, логика управления цветовой схемой сайта (Light/Dark) вынесена в отдельный файл `useTheme.js`. Этот хук инкапсулирует в себе работу с `localStorage`, манипуляцию атрибутами тега `body` и переключение состояния. Это позволяет легко внедрять функционал темизации в любой компонент приложения.

3.4 Взаимодействие с API

Для работы с сервером (Node.js/Express) используется `axios`. Настроен `request interceptor`, который автоматически добавляет заголовок `Authorization: Bearer <token>` к каждому исходящему запросу, если пользователь авторизован. Это упрощает код компонентов, избавляя от необходимости вручную передавать токены в каждой функции.

4 Листинги основного кода

4.1 Реализация Composable (useTheme.js)

```
1 import { ref, onMounted } from 'vue';
2 export function useTheme() {
3   const currentTheme = ref('light');
4   const applyTheme = (theme) => {
5     currentTheme.value = theme;
6     document.body.setAttribute('data-theme', theme);
7     document.documentElement.setAttribute('data-bs-theme', theme);
8     localStorage.setItem('site_theme', theme);
9   };
10  const toggleTheme = () => {
11    applyTheme(currentTheme.value === 'dark' ? 'light' : 'dark');
12  };
13  onMounted(() => {
14    const savedTheme = localStorage.getItem('site_theme') || 'light'
15    ;
16    applyTheme(savedTheme);
17  });
18  return { currentTheme, toggleTheme };
19 }
```

4.2 Глобальный стор авторизации (stores/auth.js)

```
1 export const useAuthStore = defineStore('auth', {
2   state: () => ({ user: null, token: null }),
3   getters: { isLoggedIn: (state) => !!state.token },
4   actions: {
```

```

5      async login(email, password) {
6          const res = await api.post('/login', { email, password });
7          this.token = res.data.accessToken;
8          this.user = res.data.user;
9      },
10     async fetchProfile() {
11         if (!this.user) return;
12         const res = await api.get(`/users/${this.user.id}`);
13         this.user = res.data;
14     }
15 },
16 persist: true // Сохранение состояния в localStorage
17 });

```

4.3 Интеграция Axios Interceptors (api/index.js)

```

1 const api = axios.create({ baseURL: 'http://127.0.0.1:3000' });
2
3 api.interceptors.request.use(config => {
4     const token = localStorage.getItem('token');
5     if (token) {
6         config.headers.Authorization = `Bearer ${token}`;
7     }
8     return config;
9 });

```

5 Заключение

В ходе выполнения лабораторной работы была проведена полная миграция веб-приложения на фреймворк Vue 3. Использование SPA-архитектуры позволило значительно повысить скорость работы интерфейса и улучшить пользовательский опыт (UX) за счет отсутствия перезагрузок страниц.

Были освоены современные практики разработки: декомпозиция интерфейса на компоненты, управление глобальным состоянием с помощью Pinia, создание переиспользуемой логики через Composables и настройка защищенного взаимодействия с REST API через Axios. Результатом работы является масштабируемое, поддерживаемое и высокопроизводительное веб-приложение.